Code Assessment
of the Onyx Chainlink ACE Integration Smart Contracts

PUBLIC

Date July 28, 2026

Produced for



By





Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	6
4	Trust Model	10
5	System Considerations	12
6	Terminology	18
7	Findings	19
8	Limitations and use of report	28

1 Executive Summary

Dear Enzyme Foundation Team,

Thank you for trusting us to help Enzyme Foundation with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Onyx Chainlink ACE Integration according to **Scope** to support you in forming an opinion on their security risks.

Enzyme Foundation integrates Chainlink ACE, an opt-in compliance layer that runs configurable on-chain policies whenever users deposit, redeem, or transfer shares. The integration wires these checks into the existing vault flows through issuance hooks and transfer validators, without changing the core issuance and transfer logic.

The most critical subjects covered in our audit are compliance enforcement, the functional correctness of the integration, and access control. Security regarding all the aforementioned subjects is high. One of the points we raised is that **cancellation paths do not invoke hooks**, which is a design choice at the current point in time. Should any policies be used that track the lifecycle state of a request, for example a recorded open request or a per-user pending amount, we recommend the code change as outlined in the respective issue.

The general subjects covered are the correctness of the hook parameter semantics, operational behavior of batched execution, and error handling. Security here is high. As the compliance layer's effectiveness depends substantially on the fund admin's configuration, the **System Considerations** section is of particular importance. Among these, we would like to highlight **the absence of generic policy composition**.

In summary, we find that the in-scope codebase provides a high level of security, assuming the **Trust Model** for this integration.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Onyx Chainlink ACE Integration repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	19 June 2026	15b81456b47ecdbb7a26735c9a2c6d8053593d6d	Initial review
2	15 July 2026	14a08f811491d7214e27c39b364c8be91a3d9d70	Refactoring
3	23 July 2026	fd6bc5cbddfd4f53b76a153829f0d2848e884495	Fixes review

For the Solidity smart contracts, the compiler version `0.8.28` was chosen.

```
src/
├── components/
│   ├── issuance-validators/
│   │   ├── ChainlinkAceIssuanceValidatorBase.sol
│   │   ├── ChainlinkAcePostIssuanceValidator.sol
│   │   ├── ChainlinkAcePreIssuanceValidator.sol
│   │   ├── PostIssuanceExtractor.sol
│   │   └── PreIssuanceExtractor.sol
│   ├── issuance/
│   │   ├── deposit-handlers/
│   │   │   ├── ERC7540LikeDepositQueue.sol
│   │   │   └── SyncDepositHandler.sol
│   │   ├── redeem-handlers/
│   │   │   └── ERC7540LikeRedeemQueue.sol
│   │   └── utils/
│   │       ├── PostIssuanceHookHandlerBase.sol
│   │       └── PreIssuanceHookHandlerBase.sol
│   └── shares-transfer-validators/
│       └── ChainlinkAceSharesTransferValidator.sol
├── infra/
│   └── chainlink-ace/
│       └── ChainlinkAcePolicyProtectedBase.sol
└── interfaces/
    ├── IPostIssuanceHook.sol
    └── IPreIssuanceHook.sol
```

Version 2 underwent a major refactoring:

```
src/
├── components/
│   ├── issuance/
│   │   ├── deposit-handlers/
│   │   │   ├── ERC7540LikeDepositQueue.sol
│   │   │   ├── SharesMintHandler.sol
```

```
        └── SyncDepositHandler.sol
    ├── hooks/
    │   └── chainlink-ace/
    │       ├── ChainlinkAceERC7540LikeDepositQueuePostExecuteDepositRequestValidator.sol
    │       ├── ChainlinkAceERC7540LikeDepositQueuePreRequestDepositValidator.sol
    │       ├── ChainlinkAceERC7540LikeRedeemQueuePostExecuteRedeemRequestValidator.sol
    │       ├── ChainlinkAceERC7540LikeRedeemQueuePreRequestRedeemValidator.sol
    │       ├── ChainlinkAceIssuanceValidatorBase.sol
    │       ├── ChainlinkAceSharesBurnHandlerPreBurnValidator.sol
    │       ├── ChainlinkAceSharesMintHandlerPreMintValidator.sol
    │       ├── ChainlinkAceSyncDepositHandlerPostDepositValidator.sol
    │       └── extractors/
    │           ├── ChainlinkAceERC7540LikeDepositQueuePostExecuteDepositRequestExtractor.sol
    │           ├── ChainlinkAceERC7540LikeDepositQueuePreRequestDepositExtractor.sol
    │           ├── ChainlinkAceERC7540LikeRedeemQueuePostExecuteRedeemRequestExtractor.sol
    │           ├── ChainlinkAceERC7540LikeRedeemQueuePreRequestRedeemExtractor.sol
    │           ├── ChainlinkAceSharesBurnHandlerPreBurnExtractor.sol
    │           ├── ChainlinkAceSharesMintHandlerPreMintExtractor.sol
    │           └── ChainlinkAceSyncDepositHandlerPostDepositExtractor.sol
    ├── interfaces/
    │   ├── IERC7540LikeDepositQueuePostExecuteDepositRequestHook.sol
    │   ├── IERC7540LikeDepositQueuePreRequestDepositHook.sol
    │   ├── IERC7540LikeRedeemQueuePostExecuteRedeemRequestHook.sol
    │   ├── IERC7540LikeRedeemQueuePreRequestRedeemHook.sol
    │   ├── ISharesBurnHandlerPreBurnHook.sol
    │   ├── ISharesMintHandlerPreMintHook.sol
    │   └── ISyncDepositHandlerPostDepositHook.sol
    └── redeem-handlers/
        ├── ERC7540LikeRedeemQueue.sol
        └── SharesBurnHandler.sol
└── shares-transfer-validators/
    ├── AddressListsSharesTransferValidator.sol
    ├── ChainlinkAceSharesTransferExtractor.sol
    └── ChainlinkAceSharesTransferValidator.sol
├── infra/
│   └── chainlink-ace/
│       └── ChainlinkAcePolicyProtectedBase.sol
├── interfaces/
│   └── ISharesTransferValidator.sol
└── shares/
    └── Shares.sol
```

2.1.1 Excluded from scope

The scope of this assessment is the additional functionality introduced in **Version 1** and **Version 2** of this assessment. Any file not explicitly listed in the file trees above is out of scope for this assessment. This includes, but is not limited to, the Chainlink ACE smart contracts provided by the `chainlink-ace` library at version `v1.1.1`. The policy engine, the reference policies, and the extractors of that library are external dependencies and were not reviewed. This assessment evaluates the integration of the in-scope contracts with the policy engine, not the correctness of any individual policy. It assumes the intended functionality is as described. Implementing additional features may require re-review of code currently in scope of this assessment.

3 System Overview

This overview describes the most recent version (`Version 2`) of the contracts as defined in the Assessment Overview.

In the findings section we use version icons to improve the readability of the report.

Enzyme Onyx is updated to add a Chainlink ACE (Automated Compliance Engine) integration. ACE is a framework that enforces configurable policies on contract actions on-chain at execution time.

The goal of the upgrade is to integrate modular, configurable compliance checks into Enzyme Onyx without changing the core issuance and transfer logic. The integration introduces policy checks at three user actions:

- **Share transfers**, when shares move between accounts.
- **Deposits**, when assets are taken in and shares are minted to the user (synchronous or asynchronous).
- **Redemptions**, when shares are burned and assets are returned to the user (asynchronous).

The deposit and redemption checks are referred to as the issuance checks. These ACE checks are independent of, and run in addition to, the optional local allowlists that the deposit handlers already maintain (see Issuance hooks and validators).

The integration comprises the following components:

- **Issuance hooks.** Optional call-out points on the issuance handlers, one dedicated hook per issuance action. When no hook is configured, issuance proceeds unchanged.
- **Issuance validators.** The contracts wired into the issuance hooks. They forward each deposit or redeem intent to a policy engine, which runs the configured policies and reverts if the action is not allowed.
- **Shares transfer validator.** A validator attached to the `Shares` contract. It forwards every share `transfer` and `transferFrom` to a policy engine, which runs the configured policies and reverts if the transfer is not allowed.
- **Extractors.** Helper contracts registered on the policy engine. They decode each issuance action into the named parameters that the policies evaluate.

As an example, a synchronous deposit flows through these layers in a single transaction. The deposit handler first applies its local depositor allowlist, then mints the shares and pulls in the assets. It then calls the configured post-deposit validator, which forwards the action to the policy engine through its `runPolicy` modifier. The engine decodes the action with the registered extractor and evaluates the ordered policies, reverting the whole deposit if a policy rejects it.

3.1 Chainlink ACE

Chainlink ACE is a framework to configure and automatically enforce policies on specific contract actions. A `PolicyEngine` contract is the entry point: a contract attaches itself to the engine and, when a protected action is triggered, calls `run()` on it. The engine looks up the extractor registered for the action's selector, which decodes the action into the named parameters the policies read, then evaluates the registered policies in a predefined order and determines whether the action may proceed.

Each policy is a standalone contract, registered per calling contract and per selector in an ordered array. For each policy the engine calls `run()`, a view function returning `Allowed`, `Continue`, or `None`, followed by `postRun()`, which may change state and runs only if `run()` did not revert. The engine short-circuits on a policy whose `run()` returns `Allowed` and lets the action proceed. If every policy returns `Continue` or `None`, the engine applies the configured default of allow or reject.

Policy logic can be arbitrary, including external calls and Chainlink DON (Decentralized Oracle Network) features, and any revert in a policy propagates and reverts the entire protected action.

3.2 Deployment and policy configuration

The integration is opt-in. A fund makes use of ACE by deploying a `PolicyEngine`, wiring the validators to it, and registering policies. The high-level flow is:

1. The fund owner deploys a `PolicyEngine` through the `PolicyEngineFactory` and is set as its `initialOwner`. The factory is permissionless and `initialOwner` is an arbitrary argument, so this owner being the Onyx fund owner is the intended setup rather than an on-chain guarantee.
2. The `initialOwner` becomes the engine admin and registers the policies for the fund's validators, per protected contract and per selector.
3. These policies run in order, whenever the issuance hooks or the transfer validator are reached in the Onyx code.

The engine admin can add and remove policies at any time and can transfer the admin role. The engine itself is a minimal-proxy clone of a fixed implementation, so its logic is not upgradeable. The engine, the factory, and the policy configuration are part of the external `chainlink-ace` library and are outside the scope of this audit.

3.3 Issuance hooks and validators

Users deposit assets and redeem shares through the respective deposit and redeem handlers. Each handler stores a dedicated hook address per issuance action and forwards the action to that hook when one is set. The admin or owner configures each hook independently, and `address(0)` disables it.

The hooks are wired into the issuance flow as follows.

- `ERC7540LikeDepositQueue` invokes a pre-request hook when a deposit request is created and a post-execute hook after the requested shares are minted.
- `ERC7540LikeRedeemQueue` invokes a pre-request hook when a redeem request is created and a post-execute hook after the redeemed assets are sent to the user.
- `SyncDepositHandler` invokes only a post-deposit hook, after the asset is transferred and the shares are minted.

Each hook receives an `_originator` (the account that called the handler) together with the accounts and amounts specific to its action. The deposit or redeem asset address is no longer passed. A hook that needs it can read `asset()` on the calling handler.

The issuance validators are the Chainlink ACE implementations of these hooks, one validator per hook. They build on `ChainlinkAcePolicyProtectedBase`, which attaches the validator to a policy engine and does not use a per-sender context, and `ChainlinkAceIssuanceValidatorBase`, which records the single handler allowed to invoke the validator. Each validator exposes its hook entrypoint under the `runPolicy` modifier, forwarding the call to the policy engine with the validator's own selector and calldata, and a matching extractor is registered per hook to decode the payload.

Independently of the hooks, the deposit handlers keep their own optional local allowlists. `SyncDepositHandler` checks a depositor allowlist and `ERC7540LikeDepositQueue` checks a deposit-controller allowlist, both configurable by the admin and both running in addition to the ACE checks in the same transaction. `ERC7540LikeRedeemQueue` has no local allowlist. Local allowlists and ACE policies may therefore be used together.

3.4 Shares transfer validator

The `Shares` contract routes every `transfer` and `transferFrom` through a shares transfer validator. The integration provides the contract `ChainlinkAceSharesTransferValidator` for this role. Like the issuance validators it inherits from `ChainlinkAcePolicyProtectedBase`.

The active validator is set by an admin through `setSharesTransferValidator` (`onlyAdminOrOwner`). This lets the admin switch from the existing `AddressListsSharesTransferValidator` to `ChainlinkAceSharesTransferValidator`. Switching is optional. Setting the validator to `address(0)` disables the check, meaning transfers proceed without any policy enforcement.

On each transfer it forwards the action to the policy engine through its `runPolicy` modifier, using the `Shares` contract as the payload `sender` and the `validateSharesTransfer` selector. The originator (the `msg.sender` of the transfer, that is the spender for `transferFrom`), the `from`, the `to`, and the amount are carried in the payload data. A dedicated `ChainlinkAceSharesTransferExtractor` decodes them into the named parameters.

3.5 Direct mint and burn handlers

Version 2 adds two handlers that let the fund issue and redeem shares directly, outside the deposit and redeem queues. They mirror subscriptions and redemptions settled off-chain, such as fiat payments. The fund mints shares to a depositor after fiat is received, and burns a holder's shares before fiat is paid back.

- `SharesMintHandler` mints shares to specified recipients via `Shares.mintFor`.
- `SharesBurnHandler` burns shares from specified holders via `Shares.burnFor`.

Both are restricted to the admin or owner. Each runs an optional pre-hook before committing, `preMint` and `preBurn` respectively, through which a Chainlink ACE validator can enforce compliance policies. The flows do not apply on-chain entry or exit fees.

3.6 Changes in Version 2

Version 2 is a major refactoring of the ACE integration. The following changes were made:

- **Library version.** The `chainlink-ace` library is now pinned to `v1.1.1`. Previously it was `v1.0.0`.
- **Base contracts.** `ChainlinkAcePolicyProtectedBase` now inherits `PolicyProtectedFlexibleUpgradeable` from the `chainlink-ace` library instead of implementing engine attachment and context storage itself. Policy invocation was moved from an explicit `PolicyEngine.run()` call that builds a `Payload` to the inherited `runPolicy` modifier, which runs the validator's own selector and calldata through the engine. Attaching or replacing the engine is now gated by `_authorizeAttachPolicyEngine`, restricted to the admin or owner.
- **Engine lifecycle.** The policy engine is now admin-replaceable and can be set to `address(0)`, which disables enforcement for that validator. Previously it was set once at init and immutable thereafter.
- **Context handling.** This integration no longer uses per-sender context, and policy inputs are now decoded from calldata by the extractors. The inherited `setContext` / `getContext` / `clearContext` functions remain callable, and the `runPolicy` path still reads the caller's stored context and forwards it to the engine. In practice the forwarded context is always empty, because the only accounts allowed to trigger a policy run are the handler (`onlyHandler`) and `Shares` (`onlyShares`), and neither sets a context. Previously, callers set and cleared a per-sender context following a three-step pattern within a single transaction.
- **Per-site hooks.** Each issuance call site now has its own dedicated hook, with its own interface, validator, and extractor, so the policy set can be tuned per site. Each hook is optional and is configured independently by the admin or owner through a dedicated setter, where `address(0)` disables it. Previously, two generic

pre- and post-issuance hooks (`PreIssuanceHookHandlerBase` / `PostIssuanceHookHandlerBase`) covered all sites. The hooks are:

  - `ERC7540LikeDepositQueue`: `preRequestDeposit` and `postExecuteDepositRequest`.
  - `ERC7540LikeRedeemQueue`: `preRequestRedeem` and `postExecuteRedeemRequest`.
  - `SyncDepositHandler`: `postDeposit`.
  - `SharesMintHandler`: `preMint`.
  - `SharesBurnHandler`: `preBurn`.

- **Hook parameters.** Hooks now carry an `_originator` (the account that called the handler) plus site-specific amounts and accounts. The asset address is no longer passed. This is by design. An extractor that needs it can read `asset()` on the payload sender, which is the handler.
- **New handlers.** `SharesMintHandler` and `SharesBurnHandler` were added. They mint and burn shares directly via `Shares.mintFor` and `Shares.burnFor`, restricted to the admin or owner, each guarded by an optional pre-hook that can enforce ACE compliance. They are intended for flows that are settled off-chain, such as fiat subscriptions and redemptions. It allows the admin to mint shares after a depositor pays fiat to the fund, and to burn a holder's shares before the fund pays fiat back. These flows do not apply on-chain entry or exit fees.
- **Transfer validation.** The shares transfer validator interface now takes an additional `_originator` parameter. The address-list and Chainlink ACE validators share this interface and remain interchangeable.

3.7 Changes in Version 3

The following changes were made in **Version 3** in response to the review. No new functionality was added.

- **Issuance hooks.** The asynchronous hooks now take `_originator`, `_controller`, and `_owner` as distinct, documented parameters. At request time `_owner` is the caller-supplied value. At execution the stored request tracks only the controller, so `request.controller` is passed in the `_owner` slot, which is correct for handlers enforcing `_controller == _owner == msg.sender` at request time.
- **Policy engine binding.** The ACE validators now extend `PolicyProtectedBaseUpgradeable`, which rejects a zero engine on both init and re-attach. Engine enforcement can no longer be unset.
- **Initializer hardening and cleanup.** `_disableInitializers()` was moved to the shared `ChainlinkAcePolicyProtectedBase` (covering every validator) alongside `SyncDepositHandler`. Clarifying and documenting NatSpec was added to the hook interfaces, and unused errors and stale NatSpec were removed.

4 Trust Model

4.1 Roles

The trust assumptions on the Onyx privileged roles are unchanged from the previous public audit of the Onyx contracts. The roles below are those introduced or relied on by the Chainlink ACE integration in the current scope.

Onyx fund admin or owner (`onlyAdminOrOwner`)

- Trust level: Fully trusted.
- Permissions specific to the ACE integration: sets the shares transfer validator (including `address(0)`, which disables transfer validation) and the per-site issuance hooks, and attaches or replaces the policy engine on each validator (including `address(0)`, which disables enforcement).
- Further permissions: mints shares to any account through `SharesMintHandler` and burns shares from any holder through `SharesBurnHandler`.
- Worst-case impact: Disable transfer validation, issuance hooks, or the policy engine to bypass all compliance checks. Mint shares to dilute holders, or burn holders' shares to confiscate or destroy them.

Policy engine `DEFAULT_ADMIN_ROLE`

- Trust level: Fully trusted.
- Permissions: The AccessControl admin role for the engine. May grant and revoke other roles, including itself.
- Worst-case impact: Grant itself `ADMIN_ROLE` and `POLICY_CONFIG_ADMIN_ROLE`, gaining the combined capabilities of both roles below and their worst-case impacts.

Policy engine `ADMIN_ROLE`

- Trust level: Fully trusted.
- Permissions: Controls which checks are enforced. Adds, positions, and removes policies, sets the extractors and policy mappers, and sets the engine-wide and per-target default-allow.
- Worst-case impact: Disable compliance by removing policies or defaulting to allow, register a malicious policy that executes inside protected actions, or block actions and DoS the protected flows. A wrong extractor may mis-decode the parameters policies act on.

Policy engine `POLICY_CONFIG_ADMIN_ROLE`

- Trust level: Fully trusted.
- Permissions: Sets versioned per-policy configuration via `setPolicyConfiguration`.
- Worst-case impact: Misconfigure a policy so it enforces the wrong constraints, thereby weakening or breaking the policy.

All three engine roles are granted to the initial owner at deployment and may later be held by different addresses. The policy engine is assumed correctly configured, honest, and available, so that the intended policies are enforced on the relevant protected actions.

4.2 General Assumptions

The assumptions below are not tied to a single role. They concern correct configuration of the policy engine and correct authoring of any custom policies, and each is detailed in a related issue or system consideration.

- **Policies are trusted code.** The engine admin registers only non-malicious, correctly implemented policies. Each policy is an external contract that may call other contracts, so a malicious or buggy policy can revert valid actions, alter shared state, or call untrusted code.
- **Policy ordering and default-allow are configured together.** Policies run in insertion order and the first to allow or revert decides the call. When no policy does, `defaultPolicyAllow` (or the per-target override) applies. The order and the effective default must match the policy set.
- **Engine topology is honestly administered.** The engine factory and `attach()` are permissionless, and the engine owner is not enforced on-chain to equal the fund owner. Each component's attached engine is assumed to be the intended one, administered by a trusted party, whether shared across funds or deployed per fund.
- **Selectors and extractors are configured correctly.** The engine resolves one extractor per 4-byte selector. Distinct actions must have distinct selectors with a matching, correct extractor. A collision or wrong extractor mis-decodes parameters.
- **Deposit and redeem assets are standard ERC20.** Each fund's underlying asset is assumed to be a plain ERC20 with no transfer callbacks or hooks, no fee on transfer, and no rebasing. A callback-bearing asset would open a reentrancy surface in the handlers, whose policy engine call runs after shares are minted or burned and assets are moved.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 A Single Failing Request Reverts the Whole Settlement Batch

System Consideration | Version 1

`ERC7540LikeDepositQueue.executeDepositRequests` and `ERC7540LikeRedeemQueue.executeRedeemRequests` process a list of request ids in a single loop with no per-request isolation. If any one request reverts, the whole transaction reverts and no request in the batch settles. The admin can recover by removing the offending id and resubmitting, but only after identifying it off chain.

The post-issuance hook added by this integration introduces a new way for a single request to revert the batch. A post-issuance policy that rejects one request, for example a sanctions or allowlist policy whose verdict changed for that receiver between request and settlement, reverts the entire loop and blocks settlement of every other request batched with it. A request that passed the pre-issuance check at request time can therefore still fail at settlement and take the rest of the batch with it. In case the policies can revert on values users could control, this would open a DoS vector.

SC2 Chainlink DON Features Require Network Support

System Consideration | Version 1

An Onyx fund owner who chooses to use Chainlink DON (decentralized oracle network) features, for example off-chain policy execution or DON-validated policies such as `CertifiedActionDONValidatorPolicy`, relies on that infrastructure being present on the chain where the fund is deployed. ACE is currently in beta, and at the time of writing its supported networks are Ethereum, Arbitrum, Avalanche, Base, and Polygon (see the Docs).

Onyx's intended networks include chains outside the supported set, namely Plume and MegaETH. A fund deployed there cannot use these features, so a compliance control assumed to depend on them would be absent for that fund. Purely on-chain policies may run, but any guarantee resting on such a feature does not hold on an unsupported chain.

Because ACE is in beta, the supported set may change, so anyone relying on these features must verify support for the target chain at deployment and over the fund's lifetime.

SC3 Custom Policy `postRun` Must Be Engine-Gated

System Consideration | Version 1

Custom policies that inherit from `Policy` and override `postRun`, or implement `IPolicy` directly, must restrict the function with the `onlyPolicyEngine` modifier. If omitted, the state the policy relies on for its checks could be altered outside the engine entirely, independent of any target configuration. An attacker could then control the policy's state, so its checks can be made to pass or fail arbitrarily.

This is separate from the namespacing of shared state by subject, which the engine likewise does not enforce.

SC4 Function Selector Collisions Are Not Detected by the Policy Engine

System Consideration | Version 1

The policy engine identifies protected actions by their four byte function selector. Extractors are registered per selector across the whole engine through `setExtractor`, and policy chains are registered per target and selector through `addPolicy`. A selector is a truncated hash of a function signature, so two different signatures can share the same four byte value, and the engine neither detects nor prevents such a collision. An extractor confirms only that the incoming selector equals the one it expects, which a colliding selector would also satisfy.

A collision might appear when a single engine is shared across additional targets or reused for other policy-protected contracts, so that its engine wide extractor mapping holds further selectors beyond the three above. In that configuration it is the admin's responsibility to ensure that no two registered selectors collide and that the extractor registered for each selector matches the calldata layout of every action that resolves to it, because the engine applies the extractor and the policy chain purely by the four byte key.

SC5 No Generic Policy Composition

System Consideration | Version 1

We provide some remarks about the logic of Chainlink ACE `PolicyEngine`, and how it influences the evaluation of multiple policies. For each protected function, an integrator configures a list of policies on the engine, keyed by target and selector, and the engine evaluates them in order to decide whether to allow or block the call.

Engine Semantics

A single policy call returns `IPolicyEngine.PolicyResult` (`Allowed`, `Continue`, or `None`), or reverts. During the loop:

1. `Allowed`: the engine returns immediately and the action is permitted. All remaining policies are skipped.
2. `Continue` (or the default `None`): the engine calls the next policy.
3. Revert: the engine catches the revert and bubbles up the error.

If the loop ends with no `Allowed` and no revert, the engine falls back to the effective default: the per-contract `targetDefaultPolicyAllow[target]` if set via `setTargetDefaultPolicyAllow`, otherwise the engine-wide `defaultPolicyAllow`.

In short, `Allowed` is a short-circuit allow, a revert is a global hard block, and `Continue` defers to the next policy or to the default. Because both `Allowed` and a revert short-circuit, a policy's position in the chain is part of its meaning, not just its presence.

Issues with Logical Composition of Policies

The meaning of `Continue` is relative to the default. Under default allow, a policy chain in which every policy returns `Continue` permits the action, so `Continue` means "no objection" and a policy blocks only by reverting. Under default reject, the same policy chain denies the action, so `Continue` means "no authorization" and the action passes only if some policy returns `Allowed`.

This allows a limited AND composition: policies that signal approval with `Continue` and reject by reverting compose in series as the logical AND of their conditions, assuming the effective default is allow. The logical OR of such policies cannot be expressed, however: rejection is signaled by reverting, and a revert is a global hard

block that no later policy can override, so the first rejecting policy ends the call. Once policies reject by reverting, any chain of them is necessarily an AND.

As a consequence, the default and the policy set are coupled and must be configured together. The bundled policies approve with `Continue` and reject by reverting, so they require default allow. Under default reject they never return `Allowed` and would block every action. Policy writers should know that a policy's logic depends on the default and on its position within the policy chain, and that complex or mixed logic belongs in a single custom policy rather than composed from smaller ones.

SC6 Policies Are Trusted External Code

System Consideration | Version 1

A policy is external code that the engine calls into during a protected action, so registering a policy grants that code execution inside Onyx's own deposit, redemption, and transfer flows. The policy admin, who is fully trusted as described in the trust model, must therefore treat every registered policy as fully trusted code and register only policies whose source has been thoroughly reviewed.

A malicious or compromised policy, or one that calls out to untrusted contracts, might:

- Approve any action by returning `Allowed`, or revert to block and brick all protected flows.
- Mutate its own state, including state shared across targets, corrupting every decision that relies on it.
- Make arbitrary external calls and grief gas.

Such a policy executes mid-transaction with no reentrancy guard, though Onyx limits the reach of reentry since privileged mutators are gated to handlers and the admin or owner and the share price is not a function of live supply.

SC7 Policy Validation Gas Scales with Batch Size × Policy Count

System Consideration | Version 1

Each executed deposit or redeem request runs the post-issuance hook, which invokes every policy configured for that selector. A batch of n requests therefore performs up to $m * n$ policy executions, where m is the per-selector policy count, capped at 8 (`MAX_POLICIES`). Both m and the batch size n are set by the admin or owner, the latter when calling `executeDepositRequests` or `executeRedeemRequests`, so neither can be inflated by users.

Batch size n is not bounded at the contract level. With many configured policies and a large batch, the all-or-nothing call can approach the block gas limit and revert without settling any request. Operators should size batches with the policy count in mind and avoid policies with high per-request cost. Note that this is an operational consideration, not an external denial-of-service risk.

SC8 Shared Stateful Policies Can Contaminate State Across Targets

System Consideration | Version 1

A single policy instance can be registered for many target and selector pairs within Chainlink ACE. The engine only rejects re-adding a policy to the same target and selector, so nothing stops one deployed policy from backing several targets at once, for example the deposit and redeem validators, or validators of different funds sharing one engine. The engine does not isolate state per target. It passes the target as the `subject` argument and leaves the policy to namespace its state by that subject. The `postRun` documentation states

that mutations should consider state relative to the target because the policy may be shared. This is a convention, not something the engine enforces.

A stateful policy that does not namespace by subject therefore shares its state across every target it backs. Activity on one target alters the accounting another relies on, through the normal authorized flows of those targets rather than any unauthorized caller. The bundled `VolumeRatePolicy` is an example: it keys its rate window by an account parameter and ignores the subject, so one instance attached to two targets merges their windows for a given account. This is most likely when one engine is shared across vaults and a stateful policy instance is reused.

SC9 Synchronous Deposits Call Hook After Settlement

System Consideration | Version 1

The `SyncDepositHandler` calls its hook only after the deposit completes, not before. In contrast, the queued deposit and redeem handlers run a pre-hook at request time and a post-hook at execution. On the synchronous path the hook fires after shares are minted and assets are pulled. Because the deposit is atomic, a reverting hook still rolls back the mint and the transfer.

Integrators writing policy logic should be aware that only the `postDepositHook` (**Version 2** nomenclature) is called on synchronous deposits.

SC10 Asset Resolution in Policies and Extractors

System Consideration | Version 2

The issuance hooks do not pass the underlying asset to the policy engine. A `Shares` vault may wrap more than one asset, and when a policy needs the asset a custom extractor is expected to obtain it by calling `asset()` on `_payload.sender`.

`_payload.sender` is the contract that invoked the validator, i.e. the handler for issuance flows or `Shares` for the transfer flow. Anyone writing extractors or policies against these hooks should note the following.

1. The ERC7540-like queues are the only senders that expose `asset()` (`ERC7540LikeIssuanceBase.asset()`). `SharesMintHandler`, `SharesBurnHandler`, and `Shares` do not, and their flows are not associated with any single asset: mint and burn settle off-chain, and transfers move shares. Calling `asset()` on a sender that lacks it reverts, which `PolicyEngine` bubbles up as `ExtractorError` and reverts the whole operation. An extractor must therefore call `asset()` only on senders that expose it, i.e. the queues. There it returns only that queue's asset, since a multi-asset vault is just several single-asset queues, a policy needing cross-asset or vault-wide logic must resolve the other queues itself.

2. An extractor is keyed only by selector (`extractorBySelector[selector]`), while a policy is keyed by target and selector (`addPolicy(target, selector, policy)`). So a USDC queue and a WETH queue that share the `postExecuteDepositRequest` selector use the same single extractor, but each can have its own policy. Asset-specific resolution therefore belongs in the policy, which is per queue and receives `payload.sender`, not in the shared extractor.

SC11 Cross-Chain Compliance Checks Apply to the `DepositorWallet`, Not the User

System Consideration | Version 1

ACE enforces vault compliance by checking the on-chain subject of a flow. When a flow originates on another chain, that subject is the `DepositorWallet`, not the end user:

- **Sync deposit:** `SyncDepositHandler.__deposit`'s post-deposit hook (`ChainlinkAceSyncDepositHandlerPostDepositValidator`) sees `originator = msg.sender`, i.e. the `DepositorWallet`.
- **Async deposit / redeem:** the `preRequestDeposit` / `preRequestRedeem` hooks evaluate `originator`, `controller`, and `owner` — all the `DepositorWallet`.
- **Share transfer:** `ChainlinkAceSharesTransferValidator.validateSharesTransfer` runs only on `Shares.transfer()` / `transferFrom()`, where one leg is always a CCIP address (token pool or relay), never the user. It is skipped on `mintFor` / `burnFor`, so a burn/mint pool bypasses it entirely; only a lock/release pool is covered.

Policies gated on the end user (blocklists, allowlists, transfer limits) must therefore treat the `DepositorWallet` as a proxy and tie it back to a real identity. What that guarantees depends on the chosen identity/verification model.

Moreover, the following limitations are inherent:

- A chain-local compliance set only sees the local leg. A destination-chain policy can gate the local release/mint, but cannot tell whether the source-chain sender is compliant.
- Per-account gating is evadable: a blocklisted party can route through a fresh wallet no list covers yet. Such policies constrain honest users more reliably than determined ones.

SC12 Admin Can Mint and Burn Shares Directly

System Consideration | Version 2

`SharesMintHandler.mint()` and `SharesBurnHandler.burn()` are `onlyAdminOrOwner` and mint shares to any account (`Shares.mintFor()`) or burn them from any holder (`Shares.burnFor()`), with only a non-zero amount required. Minting without incoming assets dilutes holders and lowers the per-share value while burning removes a holder's claim.

This is not new to the ACE integration. Note 8.1 (Admin / Owner Assumptions) of the previous public audit already records that incorrect valuation updates inflate or deflate the share price. Direct minting and burning reaches the same outcome.

The optional `preMint` / `preBurn` hooks (`address(0)` disables them) let an ACE policy constrain these paths. However, they are a compliance safety net for an honest admin, not a defense against a malicious one, since the admin is already the trust root here.

SC13 Controller Allowlist Is Checked Only at Request Time

System Consideration | Version 2

The `DepositRestriction` controller allowlist (`__validateDepositRequest`) is enforced only at request time in `requestDeposit`. At settlement, `executeDepositRequests` re-reads `request.controller` from storage

and mints to it via `shares.mintFor` without re-running the check. If the controller is removed from the allowlist, or the restriction is tightened, between request and settlement, the batch still issues shares to a now-disallowed controller.

Settlement is `onlyAdminOrOwner`, so the admin can exclude disallowed controllers from the batch, and a configured post-issuance ACE hook can re-check the controller (the extractor passes it). Neither necessarily closes the gap: the hook is optional, and batching is done by an admin who may not track allowlist changes.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

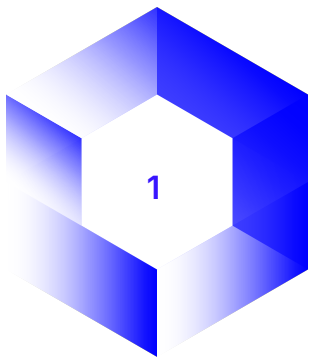
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	**Impact**		
	High	Medium	Low
High	**Critical**	**High**	**Medium**
Medium	**High**	**Medium**	**Low**
Low	**Medium**	**Low**	**Low**

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

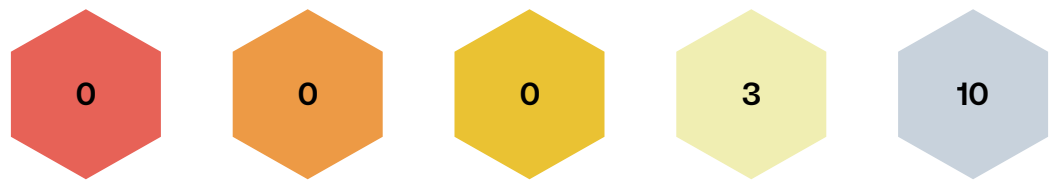
7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



Open Issues

Total number of findings



Critical High Medium Low Informational

7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Low	#001	Cancellation Paths Do Not Invoke Hooks	Risk Accepted	ⓘ
Low	#002	Policy Context Has Inconsistent Ownership Across Issuance Flows	Code Corrected	✓
Low	#003	Pre-Issuance Amount Parameter Carries Ambiguous Units Across Deposit and Redeem	Code Corrected	✓
Info	#004	`_receiver` And `_onBehalf` Parameters Collapsed in Issuance Hooks	Code Part. Corrected / Acknowledged	◐
Info	#005	Chainlink ACE Version	Code Corrected	✓
Info	#006	Document Post-Issuance Hook Amounts for Policy Authors	Code Corrected	✓
Info	#007	Implementation Contracts Do Not Disable Initializers	Code Corrected	✓
Info	#008	Policy Context Is Impractical for EOAs	Code Corrected	✓
Info	#009	Post-Issuance Hooks Run Before the Deposit Queue Settles Assets to `Shares`	Code Corrected	✓
Info	#010	Pre-Issuance Hook Runs Before the Cheaper Controller Allowlist Check	Code Corrected	✓
Info	#011	Shares Transfers Are Always Validated Under the `transfer` Selector	Code Corrected	✓
Info	#013	Disabling Policy Enforcement Does Not Emit an Event	Code Corrected	✓
Info	#014	Incomplete NatSpec and Unused Errors	Code Corrected	✓

7.2 Descriptions

#001 Cancellation Paths Do Not Invoke Hooks

Correctness	Low	Version 1	Risk Accepted

The cancellation functions do not invoke the issuance hooks. `ERC7540LikeDepositQueue.cancelDeposit` refunds the deposit asset to the controller and `ERC7540LikeRedeemQueue.cancelRedeem` returns the shares to the controller, and both emit their cancellation events, but neither calls a hook.

A policy that maintains state across the request lifecycle, for example one that records an open request or a per-user pending amount when the pre-issuance hook runs at request time, is not informed when that request is later cancelled. Its state therefore continues to reflect a request that no longer exists, and the asset or shares returned by the cancellation are not represented in any hook. Depending on the policies, this might be needed in the future.

In **Version 2** of the code, the issuance hooks are named per action and per queue.

- `ERC7540LikeDepositQueue` invokes `preRequestDeposit` at request time and `postExecuteDepositRequest` at execution.
- `ERC7540LikeRedeemQueue` invokes `preRequestRedeem` and `postExecuteRedeemRequest`.

The "pre-issuance hook" above maps to the two request-time hooks. `cancelDeposit` and `cancelRedeem` do not call any of these hooks.

> **Risk Accepted**
>
> Enzyme Foundation is aware of and accepts the residual behavior and states:
>
> > We currently don't have policies that track the requests. If we add those in the future we will take that into account, and update the code.
>
> Should such policies be added in the future, the integrator or Enzyme Foundation must take it into account and update the code accordingly.

#002 Policy Context Has Inconsistent Ownership Across Issuance Flows

Design	Low	Version 1	Code Corrected

The context that the Chainlink ACE validators read, through `getSenderContext` on `ChainlinkAcePolicyProtectedBase`, has two different owners depending on the flow. The same holds for the caller identity. Each validator builds the engine payload from one `_caller` value. That value becomes the payload `sender`, and it is also encoded into the payload `data` that the extractor later decodes into the `caller` parameter, and it keys the `getSenderContext(_caller)` lookup. Whatever address the handler reports as `_caller` is therefore what policies see as the sender, as the decoded `caller`, and as the owner of the context.

In synchronous deposits and in the request phase of the queued flows that address is the end user, so the context belongs to the user. In the settlement phase of the queued flows the handler reports the operator who executes the batch (admin or owner calling `executeDepositRequests` / `executeRedeemRequests`), so the context belongs to the operator and a single value applies to the whole settled batch.

A user therefore cannot supply context that reaches a post-issuance policy on a queued action. Depending on the policy, this divergence can enable a check to be bypassed. A policy keyed on the user's context, the caller,

or the payload sender, for example an authorization check, a per-sender rate limit, or a sanctions screen, would on the settlement path see the operator instead of the user, so the intended per-user constraint may not be enforced.

Code Corrected

The context passed to the policy engine is now always empty and no longer derived from a caller-supplied address. It is keyed on `msg.sender`, always the calling protocol contract (handler, or Shares for transfers), which never sets context. A user can only write their own context slot, which is never read.

#003 Pre-Issuance Amount Parameter Carries Ambiguous Units Across Deposit and Redeem

Design	Low	Version 1	Code Corrected

The pre-issuance hook reuses a single amount parameter for two different units. `IPreIssuanceHook` documents `_amount` as the asset amount for deposit requests and the shares amount for redeem requests. The deposit queue passes the asset amount and the redeem queue passes the shares amount, yet both call the same `preIssuance` function and therefore the same selector. The engine resolves the extractor by selector, so the single `PreIssuanceExtractor` decodes both cases and labels the value under one key, the hash of `amount`, with nothing in the decoded parameters indicating the unit.

The unit is distinguished only by which validator a policy is attached to, the deposit or the redeem pre-issuance validator, not by the data the policy receives. A unit-sensitive policy attached to both targets with one threshold compares an asset-denominated limit against a share-denominated quantity in one flow, enforcing the wrong bound. No on-chain check separates the two meanings, so the distinction rests entirely on correct configuration by the policy admin. The `PostIssuanceExtractor` exposes its quantities under separate keys for the shares amount and the asset amount, so this disambiguation already exists within the same integration.

Code Corrected

Different hooks have been implemented for different functions. In particular, the single pre-issuance hook has been split into `preRequestDeposit` (`IERC7540LikeDepositQueuePreRequestDepositHook`), which carries an `_assetAmount`, and `preRequestRedeem` (`IERC7540LikeRedeemQueuePreRequestRedeemHook`), which carries a `_sharesAmount`. The two now have distinct selectors resolved by distinct extractors, and each extractor labels the value under a unit-specific key, `keccak256("assetAmount")` for deposits and `keccak256("sharesAmount")` for redeems. The decoded parameters therefore identify the unit, so a policy no longer relies solely on which target it is attached to.

#004 `_receiver` And `_onBehalf` Parameters Collapsed in Issuance Hooks

Informational	Version 1	Code Partially Corrected / Acknowledged

`IPreIssuanceHook` and `IPostIssuanceHook` have separate `_receiver` and `_onBehalf` parameters, but every call site passes the same value for both:

- `SyncDepositHandler.__deposit()`: both `_depositor`.
- `ERC7540LikeDepositQueue` (request and execute): both `request.controller`.
- `ERC7540LikeRedeemQueue` (request and execute): both `request.controller`.

We understand this is intentional and enforced today, but it might cause problems if a later version lets the values differ. At this point, existing policies might apply their checks to the wrong account.

Code Partially Corrected

The generic `IPreIssuanceHook` and `IPostIssuanceHook` interfaces were removed and each call site now has a dedicated hook. The collapsed `_receiver` / `_onBehalf` pair no longer exists.

The four asynchronous hooks (`preRequestDeposit()` , `preRequestRedeem()` , `postExecuteDepositRequest()` , `postExecuteRedeemRequest()`) all pass `_originator` , `_controller` , and `_owner` as distinct parameters. `postDeposit()` on the synchronous path carries only `_originator` , the sole role there.

Acknowledged

The asynchronous deposit and redeem handlers enforce `_controller == _owner == msg.sender` at request time, with the comment:

```
/// @dev _controller, _owner, and msg.sender must all be the same. Support for distinct
    values may be added later.
```

At request time (`preRequestDeposit()` , `preRequestRedeem()`), `_owner` is the genuine caller-supplied value. At execute time (`postExecuteDepositRequest()` , `postExecuteRedeemRequest()`), `_owner` is a parameter of the hook, but the queue's stored request only tracks `controller` , not the request-time `_owner` , so the value passed is `request.controller` rather than the actual owner.

This keeps the interface itself flexible, but not the implementation. If a later version lets `_owner` diverge from `_controller` , the execute-time hook would still silently report the wrong account, since the real owner is never persisted anywhere between request and execution.

Enzyme Foundation acknowledges this and states:

> Very likely we will not need different owner than controller in the future, and for now code logic holds.

#005 Chainlink ACE Version

Informational	Version 1	Code Corrected

The integration pins the Chainlink ACE dependency to `v1.0.0` (commit `288c393a`). A newer release, `v1.1.1` (commit `46738579` , released 2026-06-05), is available. The pinned version is therefore one minor release behind, and `v1.1.1` changes behavior that this integration relies on. The differences are not limited to fixes, and the newer version is not a drop-in replacement because it alters interfaces the integration depends on.

The notable changes between the two versions include the following:

- The engine now enforces attachment. In `v1.0.0` , `PolicyEngine.run` and `check` do not consult the attachment state of the calling target. In `v1.1.1` , both revert with `TargetNotAttached` when the caller is not attached, so attachment becomes a hard precondition rather than bookkeeping.
- The engine became upgradeable. In `v1.0.0` , `PolicyEngine` is only `Initializable` . In `v1.1.1` , it is `UUPSUpgradeable` with `_authorizeUpgrade` restricted to the engine administrator role, and it adds an `upgradePolicy` entry point for upgrading policies through the engine.
- The policy interface changed. The `IPolicy` hooks `onInstall` and `onUninstall` gained a `subject` parameter, changing from a single selector argument to a target and selector pair. This is a breaking change for any policy written against the older interface.

- The protected base was restructured. `v1.1.1` splits `PolicyProtected` into a base and flexible and upgradeable variants, including an inline context path, where `v1.0.0` exposes only the single `PolicyProtected` and its upgradeable form.

> **Code Corrected**
>
> The Chainlink ACE dependency was updated to `v1.1.1`, the current release.

#006 Document Post-Issuance Hook Amounts for Policy Authors

Informational	Version 1	Code Corrected

The vault can charge two fees: an entrance fee on deposit and an exit fee on redeem.

The hook `IPostIssuanceHook.postIssuance()` is called from three places: `SyncDepositHandler.__deposit()`, `ERC7540LikeDepositQueue.executeDepositRequests()`, and `ERC7540LikeRedeemQueue.executeRedeemRequests()`. Parameters `_sharesAmount` and `_assetAmount` carry gross or net amounts depending on the call location.

- On deposit, `_sharesAmount` is net (the shares left after the entrance fee) and `_assetAmount` is gross (the full assets paid in).
- On redeem, `_sharesAmount` is gross (the full shares burned) and `_assetAmount` is net (the assets left after the exit fee).

In both cases the amounts are the real quantities exchanged with the user over the request: the shares they gave up or received, and the assets they paid or got back.

Because the gross-versus-net convention flips between the deposit and redeem paths and is not documented in the interface, integrators and policy authors can easily confuse the passed values. A policy that pairs the two amounts, for example to derive a per-share price or a transacted value, would compute a result skewed by the fee rate, and skewed in opposite directions on deposit versus redeem. Explicit documentation of the unit and gross/net meaning of each parameter per call site in `IPostIssuanceHook` would benefit integrators and policy authors.

> **Code Corrected**
>
> The single `IPostIssuanceHook.postIssuance()` was split into per-action hooks (`postDeposit()`, `postExecuteDepositRequest()`, `postExecuteRedeemRequest()`), each carrying one fixed convention. NatSpec on every hook interface now documents the unit and gross/net meaning of each parameter: on the deposit hooks `_sharesAmount` is net (after entrance fee) and `_assetAmount` is gross. On the redeem hook `_sharesAmount` is gross and `_assetAmount` is net (after exit fee). The cross-call-site ambiguity is removed.

#007 Implementation Contracts Do Not Disable Initializers

Informational	Version 1	Code Corrected

Several in-scope contracts use OpenZeppelin's `Initializable` but never call `_disableInitializers()` in their constructor, leaving the shared implementation behind each proxy openly initializable. `ChainlinkAcePolicyProtectedBase` and `ChainlinkAceIssuanceValidatorBase` only verify their ERC-7201 storage location in the constructor, and the concrete contracts inheriting them, `ChainlinkAcePreIssuanceValidator`, `ChainlinkAcePostIssuanceValidator`, `ChainlinkAceSharesTransferValidator` and `SyncDepositHandler`, add no `_disableInitializers()` of

their own. Any account can therefore call `init` directly on an implementation. For the validators this writes the engine (and, for the issuance validators, the handler) and calls `attach()` on an engine of the caller's choosing, emitting `PolicyEngineAttached` from the implementation. For `SyncDepositHandler` it only sets the asset and emits `AssetSet`.

The impact is confined to the implementation. Each instance runs as a proxy with its own storage and never reads the implementation's storage, and the implementation has no upgrade authority and no selfdestruct path, so initializing it cannot alter any live proxy's bindings or behavior. What remains is a stray, already-initialized implementation and spurious events. The pattern is also applied inconsistently. For example, `DepositorWallet` disables initializers in its constructor while the in-scope contracts do not.

Code Corrected

`_disableInitializers()` was added to the constructor of `ChainlinkAcePolicyProtectedBase`, the shared base now inherited by `ChainlinkAceIssuanceValidatorBase` (and every issuance validator built on it) and by `ChainlinkAceSharesTransferValidator`, and to `SyncDepositHandler`'s own constructor. Every previously affected implementation is now disabled from direct initialization.

#008 Policy Context Is Impractical for EOAs

Informational	Version 1	Code Corrected

The context channel that the Chainlink ACE validators expose, through `setContext` and `getSenderContext` on `ChainlinkAcePolicyProtectedBase`, is meant to let a caller pass transaction-specific data into policy evaluation. As built, it is impractical to use safely from an externally owned account, and it does not clear a value after that value is consumed.

Supplying context safely requires the caller to set the context, trigger the protected action, and overwrite the context within one transaction, since the channel stores the value in persistent per-sender storage rather than carrying it inline with the action. An externally owned account cannot bundle these calls on its own, so an ordinary wallet cannot follow this sequence without deploying a helper contract that performs the steps in a single call, or relying on an account abstraction mechanism such as EIP-7702. Both are cumbersome, which leaves the channel hard to use for the very users who are expected to provide context.

The channel also does not clear a value after it is consumed, which contradicts the `IPolicyProtected.setContext` documentation and departs from the reference implementation of the `PolicyProtected` base, in which `runPolicy` clears the context after each run and `runPolicyWithContext` carries it inline without persisting it.

Code Corrected

The context field is no longer set by the integrating contracts, so no user-specific context reaches policy evaluation. This removes the EOA usability concern. Moreover, the contracts now inherit `runPolicy` from `PolicyProtectedBaseUpgradeable` (through `PolicyProtectedFlexibleUpgradeable`), whose `_runPolicyAfter()` calls `clearContext()` after every run. Thus, even if the context channel were used in future versions of this integration, a set value is cleared after each run, matching the `IPolicyProtected.setContext` documentation.

#009 Post-Issuance Hooks Run Before the Deposit Queue Settles Assets to `Shares`

Informational	Version 1	Code Corrected

In `ERC7540LikeDepositQueue.executeDepositRequests`, shares are minted and the post-issuance hook is run for each request inside the processing loop, while the corresponding deposit assets are still held by the queue. The accumulated assets are moved into the `Shares` contract only once, after the whole batch has been processed. This batch transfer is a gas optimization, a single transfer instead of one per request, but it means that throughout the loop the `Shares` contract holds the newly minted shares without their backing assets, as the queue still holds those assets. The settled, consistent state exists only after the loop completes.

A policy invoked by the hook during the loop that derives state from the asset or share balances of the `Shares` or queue contracts can therefore read values inconsistent with the amounts passed to the hook, since the minted shares already exist while the backing assets are not yet credited. Such policies must not rely on the balances of `Shares` or of the queue, and should instead use the hook parameters, which carry the correct amounts for the action.

Code Corrected

Asset transfers now happen per `requestId` inside the loop, before the hook is called. A policy in the hook therefore reads a consistent state.

#010 Pre-Issuance Hook Runs Before the Cheaper Controller Allowlist Check

Informational	Version 1	Code Corrected

In `ERC7540LikeDepositQueue.__requestDeposit`, the pre-issuance hook runs before the controller allowlist check in `__validateDepositRequest`. The hook is an external call into the policy engine that runs the extractor and the configured policy chain, which is considerably more expensive than the allowlist check, which is an internal lookup or a single external view call. Because the more expensive check runs first, a request whose controller the allowlist would reject still pays for the full policy engine run before the transaction reverts.

Code Corrected

`__validateDepositRequest` now runs before the policy hook invocation.

#011 Shares Transfers Are Always Validated Under the `transfer` Selector

Informational	Version 1	Code Corrected

`validateSharesTransfer` always builds the engine payload with the ERC20 `transfer` selector and the sender set to the debited owner, regardless of whether the originating call was `transfer` or `transferFrom`. `Shares` routes both overrides through this one function, and for `transferFrom` it forwards the token owner (`_from`), not the spender that initiated the call. The spender is therefore dropped before the validator and

never reaches the payload. The upstream `ERC20TransferExtractor` resolves the originator from the payload sender under the `transfer` selector, so it reports the owner as the originator for both flows.

A consequence is that policies cannot gate on the initiator of a `transferFrom`, since only the owner and the recipient are presented. Both of those are still validated, so a restricted owner or recipient cannot bypass enforcement through this path. Policies therefore cannot perform spender-level gating.

Code Corrected

`validateSharesTransfer()` now takes an `_originator` parameter: the debited owner for `transfer`, and the initiating spender for `transferFrom`, passed alongside `_from` and `_to`. A dedicated `ChainlinkAceSharesTransferExtractor`, keyed on the `validateSharesTransfer` selector, passes `originator`, `from`, `to`, and `sharesAmount` to policies. The `ERC20TransferExtractor` and reused `transfer` selector are no longer involved.

Policies can now gate on the initiator of a `transferFrom`. Both the `transfer` and `transferFrom` flow still route through one function under a single selector, so a policy cannot tell them apart directly. However, the only case where this collapses is a `transferFrom` where the spender equals the owner, which is equivalent to a plain `transfer`.

#013 Disabling Policy Enforcement Does Not Emit an Event

Informational | **Version 2** | **Code Corrected**

Setting a validator's policy engine to `address(0)` disables enforcement. The `PolicyProtectedFlexibleUpgradeable` base (from the `chainlink-ace` library) allows this by removing the non-zero check, and `_attachPolicyEngine` emits `PolicyEngineAttached` only for a non-zero engine. The old engine's `detach()` emits `TargetDetached` on the engine contract, but the validator itself emits nothing when its engine is unset. Off-chain monitoring watching the validator therefore cannot detect an admin disabling checks.

Code Corrected

`PolicyProtectedBaseUpgradeable` is used instead, which does not allow the engine to be set to the zero address. The missing-event scenario no longer arises.

#014 Incomplete NatSpec and Unused Errors

Informational | **Version 2** | **Code Corrected**

The following is a non-exhaustive list of observations on documentation and unused code:

- **Hook interfaces: undocumented parameters.** The following hook interfaces do not provide documentation for their parameters, even though they carry meanings that vary by hook. `_sharesAmount` and `_assetAmount` are net or gross depending on the leg, and `_originator` is the end user on some hooks and the admin or owner on others.
  - `IERC7540LikeDepositQueuePreRequestDepositHook.preRequestDeposit`
  - `IERC7540LikeDepositQueuePostExecuteDepositRequestHook.postExecuteDepositRequest`
  - `IERC7540LikeRedeemQueuePreRequestRedeemHook.preRequestRedeem`
  - `IERC7540LikeRedeemQueuePostExecuteRedeemRequestHook.postExecuteRedeemRequest`
  - `ISyncDepositHandlerPostDepositHook.postDeposit`

- `ISharesMintHandlerPreMintHook.preMint`
- `ISharesBurnHandlerPreBurnHook.preBurn`

- **`AddressListsSharesTransferValidator.sol`: incorrect NatSpec.** The `@param recipientAllowlist`, described in the NatSpec of `SharesTransferValidatorStorage`, has been renamed.
- **`Shares.sol`:** `Shares__AddAllowedHolder__AlreadyAdded`, `Shares__GetDepositAssetsDest__NotSet`, `Shares__RemoveAllowedHolder__AlreadyRemoved`, and `Shares__ValidateTransferRecipient__NotAllowed` are declared but never used.

Code Corrected

All items were addressed. The hook interfaces now carry `@param` NatSpec documenting `_originator`, the account roles, and the gross/net meaning of the amount parameters. The incorrect NatSpec was fixed and the unused errors were deleted.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.